

Web and Application Security

Chapter III

Web application vulnerabilities

- A web application vulnerability is a security flaw, weakness, or misconfiguration in a web application's code, design, or implementation that can be exploited by attackers to gain unauthorized access, steal sensitive data (e.g., credit card info), or disrupt services.
- These flaws, often stemming from poor input validation, can lead to serious risks like data breaches, malware installation, and user session hijacking.

Common Examples of Web Application Vulnerabilities

- **Broken Access Control** (e.g., IDOR): Attackers bypass authorization to access unauthorized data, such as changing URL IDs to view other users' profiles.
- **Injection Attacks** (e.g., SQLi): Malicious SQL code is inserted into input fields, allowing attackers to manipulate or delete backend database content.
- **Cross-Site Scripting (XSS)**: Malicious scripts are injected into web pages and executed by innocent users' browsers, often leading to session hijacking.
- **Security Misconfigurations**: Unsecured default settings, exposed administrative interfaces, or open cloud storage, which can leak sensitive information.
- **Cross-Site Request Forgery (CSRF)**: Tricks a authenticated user's browser into performing unwanted actions on a different website without their consent.

SQL Injection

- SQL Injection is a security flaw in web applications where attackers insert harmful SQL code through user input.
- This can allow them to access sensitive data, change database contents, or even take control of the system.
- It's important to know about SQL Injection to keep web applications secure.
- SQL Injection (SQLi) is a security vulnerability that occurs when an attacker can manipulate a web application's database queries by inserting malicious SQL code into user input fields.
- These injected queries can manipulate the underlying database to retrieve, modify, or delete sensitive data. In some cases, attackers can even escalate privileges, gaining full control over the database or server.

Types of SQL Injection

- **Classic SQL Injection:** The hacker directly adds bad commands into a database query to make it do things it shouldn't.
- **Union-Based SQL Injection:** The hacker uses the **UNION** command to get information from other tables in the database.
- **Error-Based SQL Injection:** The hacker looks at error messages from the database to learn secret information about it.
- **Blind SQL Injection:** The hacker can't see error messages, so they guess information by watching how the website behaves (like true/false responses, time delays, etc.).

- `<?php // Vulnerable code $username = $_POST['username']; $password = $_POST['password']; $query = "SELECT * FROM USERS WHERE USERNAME = '$username' AND PASSWORD = '$password'"; $result = mysqli_query($conn, $query); ?>`

- If a malicious user enters ' **OR '1'='1** as the password, the query becomes:
- `SELECT * FROM USERS WHERE USERNAME = 'admin' AND PASSWORD = " OR '1'='1';`
- This condition is always true because **'1'='1'** evaluates to true, allowing the attacker to log in without knowing the actual password.

Error-Based SQL Injection

```
CREATE TABLE CUSTOMERS (  
    ID INT NOT NULL,  
    NAME VARCHAR (20) NOT NULL,  
    AGE INT NOT NULL,  
    ADDRESS CHAR (25),  
    SALARY DECIMAL (18, 2),  
    PRIMARY KEY (ID)  
);
```

```
INSERT INTO CUSTOMERS VALUES  
(1, 'Ramesh', 32, 'Ahmedabad', 2000.00 ),  
(2, 'Khilan', 25, 'Delhi', 1500.00 ),  
(3, 'Kaushik', 23, 'Kota', 2000.00 ),  
(4, 'Chaitali', 25, 'Mumbai', 6500.00 ),  
(5, 'Hardik', 27, 'Bhopal', 8500.00 ),  
(6, 'Komal', 22, 'Hyderabad', 4500.00 ),  
(7, 'Muffy', 24, 'Indore', 10000.00 );
```

ID	NAME	AGE	ADDRESS	SALARY
1	Ramesh	32	Ahmedabad	2000.00
2	Khilan	25	Delhi	1500.00
3	Kaushik	23	Kota	2000.00
4	Chaitali	25	Mumbai	6500.00
5	Hardik	27	Bhopal	8500.00
6	Komal	22	Hyderabad	4500.00
7	Muffy	24	Indore	10000.00

Imagine a website uses this query to get a customer's name and salary by ID:

```
SELECT NAME, SALARY FROM CUSTOMERS WHERE ID = '1';
```

If a hacker types **1 OR 1=1** instead of a normal ID, the query becomes:

```
SELECT NAME, SALARY FROM CUSTOMERS WHERE ID = 1 OR 1=1;
```

Because **1=1** is always true, the database returns **all rows** instead of just one record, exposing everyone's information:

ID	NAME	AGE	ADDRESS	SALARY
1	Ramesh	32	Ahmedabad	2000.00
2	Khilan	25	Delhi	1500.00
3	Kaushik	23	Kota	2000.00
4	Chaitali	25	Mumbai	6500.00
5	Hardik	27	Bhopal	8500.00
6	Komal	22	Hyderabad	4500.00
7	Muffy	24	Indore	10000.00

Union-Based SQL Injection

- `SELECT name, price FROM products
UNION
SELECT columnA, columnB FROM
another_table;`

How to Prevent SQL Injection

- **Use Prepared Statements (Parameterized Queries):** Make sure user input is treated only as data, not as part of the database commands.
- **Use Stored Procedures:** Keep SQL code inside the database instead of building queries with user input.
- **Validate and Clean Input:** Always check and clean what users type before using it in queries.
- **Give Minimum Permissions:** Give database accounts only the access they really need.
- **Use ORM Libraries:** Tools like Hibernate or Entity Framework handle SQL safely for you.
- **Turn Off Detailed Error Messages:** Don't show database errors to users, so hackers can't learn about the database.

- **<?php** // Safe code using prepared statements
\$stmt = \$conn->prepare("SELECT * FROM
USERS WHERE USERNAME = ? AND
PASSWORD = ?"); \$stmt->bind_param("ss",
\$username, \$password); \$stmt->execute();
\$result = \$stmt->get_result(); **?>**

Cross-site scripting (XSS)

- Cross-site scripting (also known as XSS) is a web security vulnerability that allows an attacker to compromise the interactions that users have with a vulnerable application.
- It allows an attacker to avoid the same origin policy, which is designed to segregate different websites from each other.
- Cross-site scripting vulnerabilities normally allow an attacker to masquerade as a victim user, to carry out any actions that the user is able to perform, and to access any of the user's data.
- If the victim user has privileged access within the application, then the attacker might be able to gain full control over all of the application's functionality and data.

How does XSS work?

- Cross-site scripting works by manipulating a vulnerable web site so that it returns malicious JavaScript to users. When the malicious code executes inside a victim's browser, the attacker can fully compromise their interaction with the application.

What are the types of XSS attacks?

- [Reflected XSS](#), where the malicious script comes from the current HTTP request.
- [Stored XSS](#), where the malicious script comes from the website's database.
- [DOM-based XSS](#), where the vulnerability exists in client-side code rather than server-side code

Reflected cross-site scripting

- Reflected XSS is the simplest variety of cross-site scripting. It arises when an application receives data in an HTTP request and includes that data within the immediate response in an unsafe way.
- Here is a simple example of a reflected XSS vulnerability:
- [`https://insecure-website.com/status?message=All+is+well.`](https://insecure-website.com/status?message=All+is+well.) `<p>Status: All is well.</p>`

- The application doesn't perform any other processing of the data, so an attacker can easily construct an attack like this:
- `https://insecure-website.com/status?message=<script>/*+Bad+stuff+here...+*/</script><p>Status: <script>/* Bad stuff here...*/</script></p>`

- If the user visits the URL constructed by the attacker, then the attacker's script executes in the user's browser, in the context of that user's session with the application.
- At that point, the script can carry out any action, and retrieve any data, to which the user has access.

Stored cross-site scripting

- Stored XSS (also known as persistent or second-order XSS) arises when an application receives data from an untrusted source and includes that data within its later HTTP responses in an unsafe way.
- Here is a simple example of a stored XSS vulnerability. A message board application lets users submit messages, which are displayed to other users:
- `<p>Hello, this is my message!</p>`
- The application doesn't perform any other processing of the data, so an attacker can easily send a message that attacks other users:
- `<p><script>/* Bad stuff here... */</script></p>`

DOM-based XSS

- DOM(Document Object Model)-based XSS (also known as DOM XSS) arises when an application contains some client-side JavaScript that processes data from an untrusted source in an unsafe way, usually by writing the data back to the DOM.

- In the following example, an application uses some JavaScript to read the value from an input field and write that value to an element within the HTML:
- `var search = document.getElementById('search').value; var results = document.getElementById('results'); results.innerHTML = 'You searched for: ' + search;`
- If the attacker can control the value of the input field, they can easily construct a malicious value that causes their own script to execute:
- You searched for: `<img src=1 onerror='/* Bad stuff here... */'>`

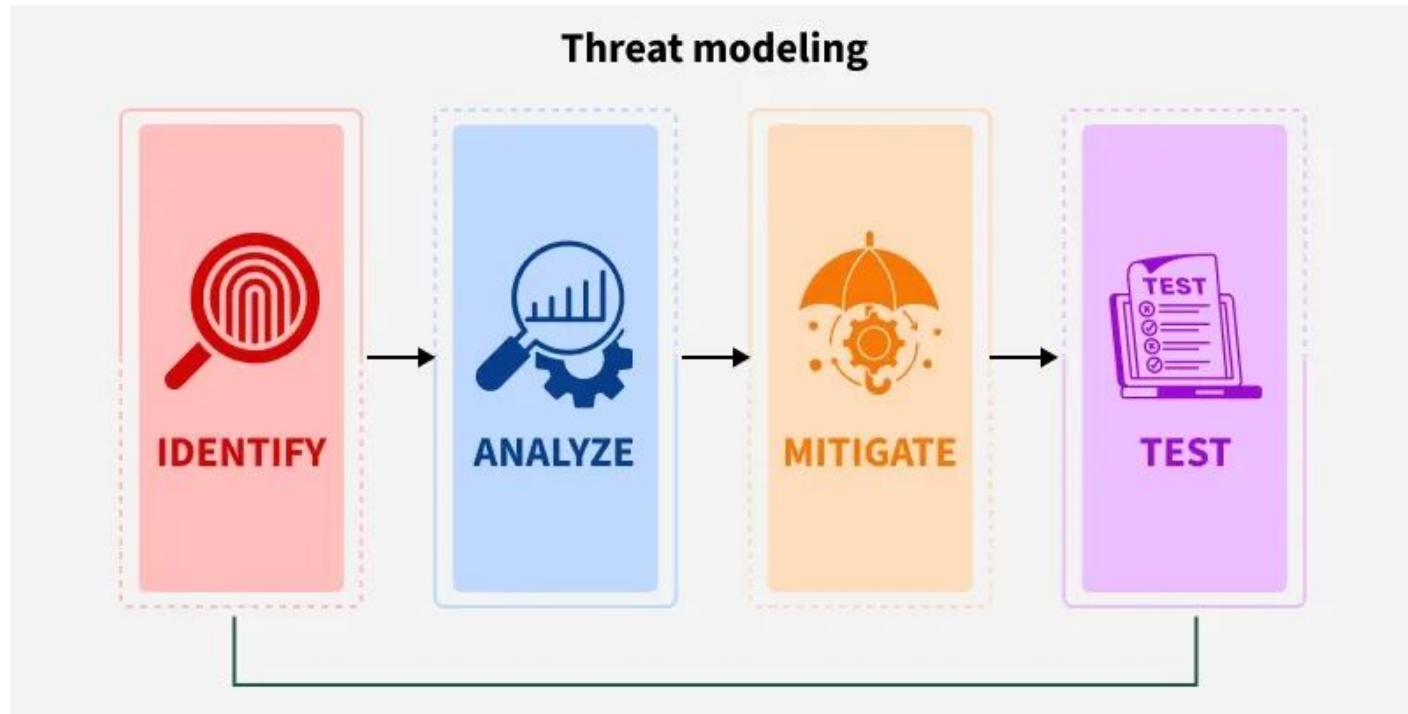
- In a typical case, the input field would be populated from part of the HTTP request, such as a URL query string parameter, allowing the attacker to deliver an attack using a malicious URL, in the same manner as reflected XSS.
-

Cross-site request forgery

- Cross-site request forgery (also known as CSRF) is a web security vulnerability that allows an attacker to induce users to perform actions that they do not intend to perform. It allows an attacker to partly circumvent the same origin policy, which is designed to prevent different websites from interfering with each other.

Threat Modelling

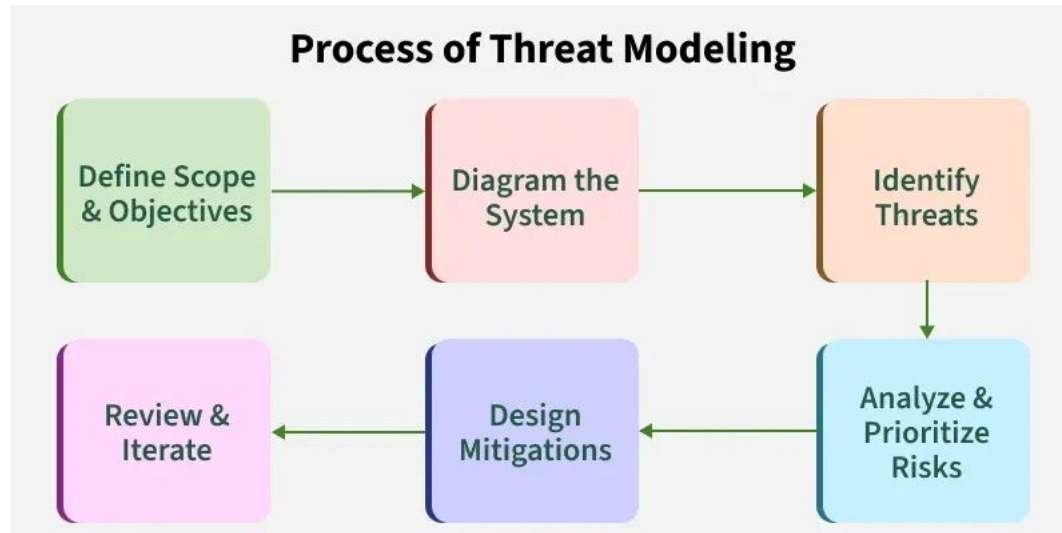
- Threat modelling is a structured method to identify, analyze, and mitigate potential threats in systems, applications, or organizations.
- It helps teams uncover vulnerabilities early and build security into the design before attackers exploit weaknesses.
- Identifies what can go wrong before deployment
- Helps understand attacker goals, techniques, and entry points
- Prioritizes risks based on impact and likelihood
- Ensures security becomes part of the development lifecycle



It can be applied to a wide range of targets such as:

- Software and applications
- Systems and networks
- Distributed systems and IoT devices
- Business processes

Process of Threat Modeling



- **1. Define Scope & Objectives**
- Clarify what part of the system you're analyzing and why.
- Identifies what needs protection
- Defines assets, boundaries, and goals
- Establishes who makes security decisions
- **2. Diagram the System**
- Create a visual map of components and data flows.
- Shows how users and systems interact
- Highlights data movement and trust boundaries
- Exposes areas where attacks may occur
- **3. Identify Threats**
- Determine what can go wrong in the system.
- Reveals attacker entry points and techniques
- Uses STRIDE or similar frameworks
- Maps threats to assets and workflows

- **4. Analyze & Prioritize Risks**

- Rank threats based on impact and likelihood.
- Focuses attention on critical risks
- Helps estimate real-world damage potential
- Supports informed decision-making

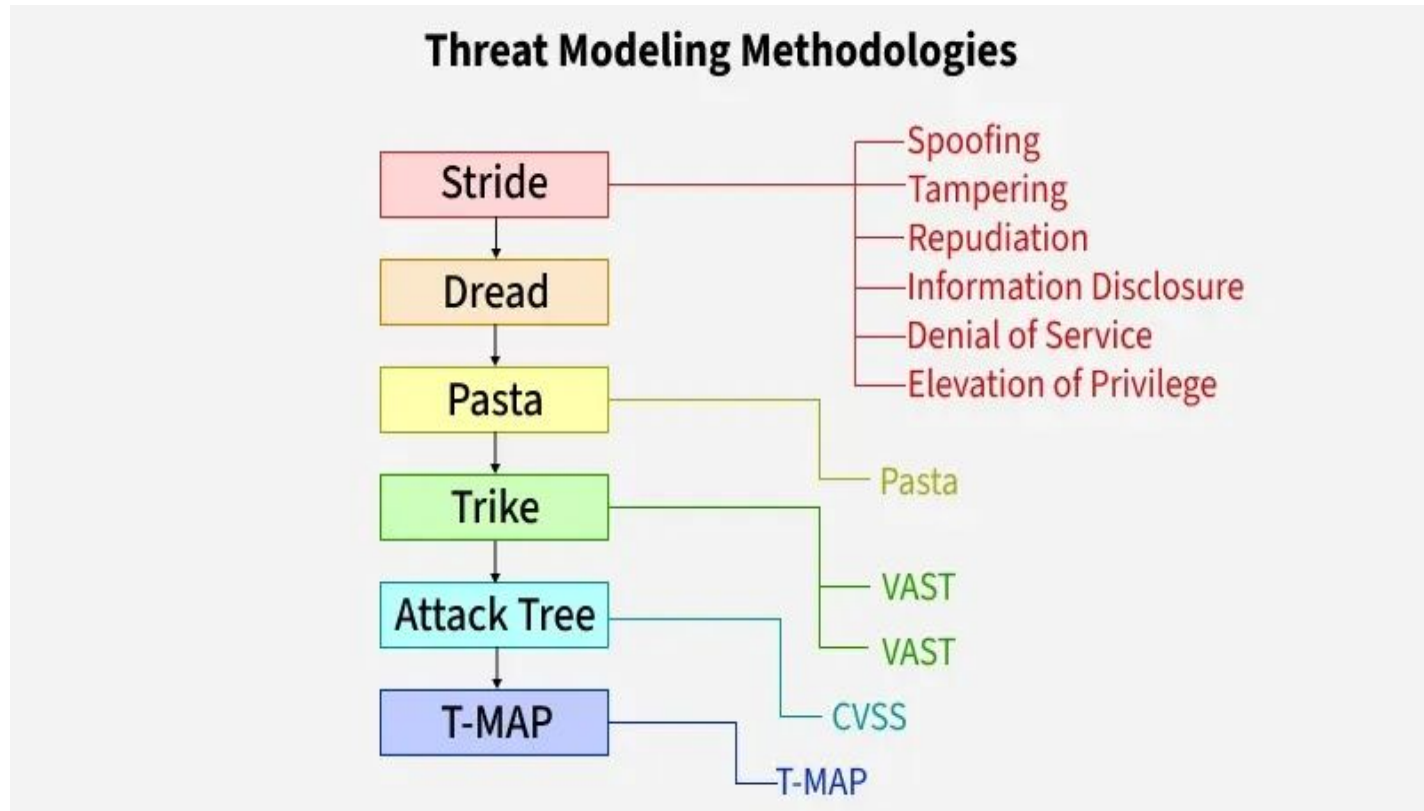
- **5. Design Mitigations**

- Plan security measures to reduce or eliminate threats.
- Defines clear mitigation actions
- Aligns controls to specific threats
- Strengthens overall system resilience

- **6. Review & Iterate**

- Continuously refine the model as the system evolves.
- Validates existing controls
- Updates diagrams as architecture changes
- Ensures ongoing security alignment

Threat Modelling Methodologies



- **STRIDE**

- A Microsoft model that categorizes threats into six major security areas.
- Spoofing → pretending to be someone/something else
- Tampering → altering data or system components
- Repudiation → denying actions without evidence
- Information Disclosure → unauthorized data exposure
- Denial of Service → making services unavailable
- Elevation of Privilege → gaining higher-than-allowed permissions

- **DREAD**

- A risk-rating system used to score threats based on severity and impact.
- Damage Potential → how severe the impact is
- Reproducibility → how easily the attack can be repeated
- Exploitability → difficulty of launching the attack
- Affected Users → number of users impacted
- Discoverability → how easy it is to find the weakness

- **PASTA (Process for Attack Simulation and Threat Analysis)**
- A 7-stage methodology focused on attacker behavior and real-world attack scenarios.
- Models threats from attacker's perspective
- Aligns system architecture to possible attack paths
- Helps design strong, risk-based security controls

- **Trike**
- A risk-management–oriented model that defines acceptable risk levels for assets.
- Uses stakeholder requirements
- Maps actions, roles, and asset permissions
- Prioritizes threats based on defined risk boundaries

- **VAST (Visual, Agile, and Simple Threat Modeling)**
- A scalable approach designed for large enterprises and agile teams.
- Uses visual models for applications and infrastructure
- Integrates easily into development workflows
- Does not require deep security knowledge
- **6. Attack Tree**
- A visual diagram showing all possible ways an attacker can reach a goal.
- Hierarchical structure with AND/OR logic
- Helps analyze multiple attack paths
- Supports structured reasoning about attack feasibility

- **7. CVSS (Common Vulnerability Scoring System)**
- A standardized scoring method to rate vulnerability severity (0–10).
- Uses metrics like impact, exploitability, complexity
- Helps prioritize vulnerability patching
- Offers consistent, industry-wide scoring
- **8. T-MAP**
- A modeling method used for COTS systems using UML diagrams.
- Identifies asset vulnerabilities and attack paths
- Calculates risk based on asset–threat relationships
- Supports structured evaluation for packaged systems

Tools for Threat Modelling

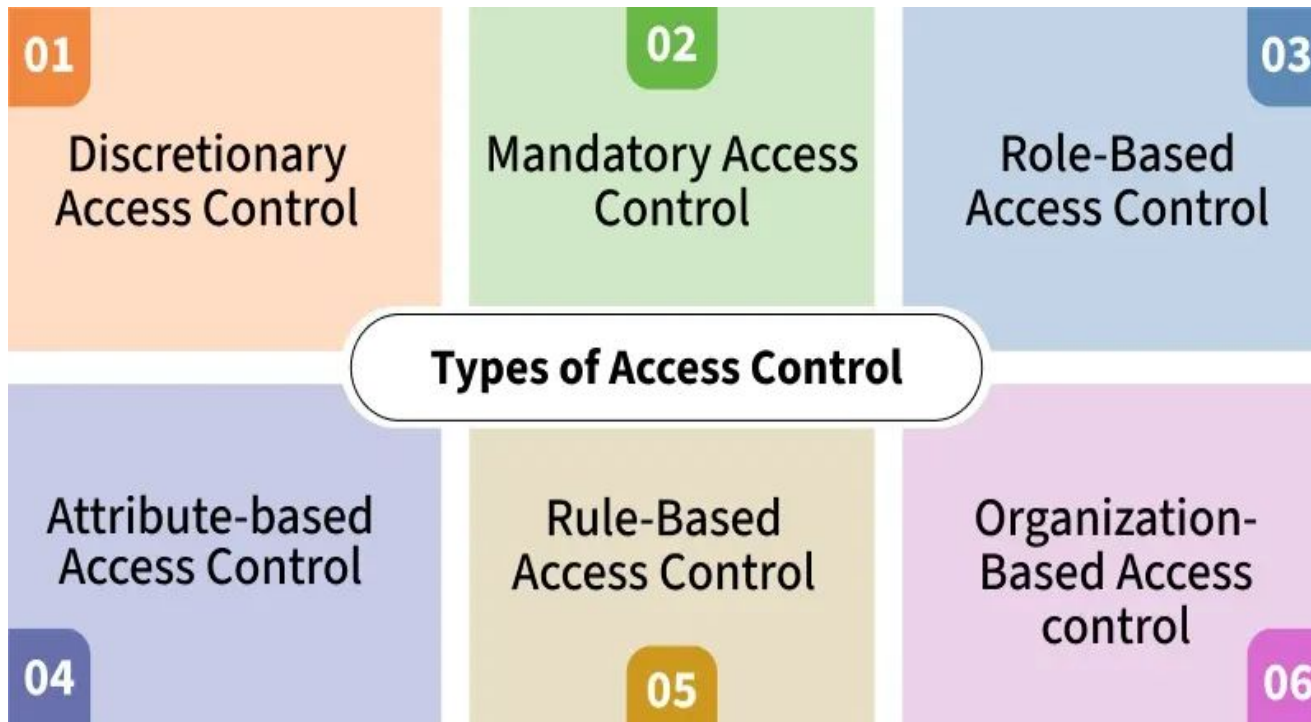
These tools help automate and streamline the threat modeling process, enabling teams to identify, assess, and mitigate security risks more efficiently throughout the software development lifecycle.

- Microsoft's Threat Modelling Tool
- MyAppSecurity
- IriuRisk
- securiCAD
- SD Elements by Security Compass
- Modeling Attack Trees
- CVSS 3.0
- Tiramisu

Access Control

- Access control is a security mechanism that defines who can access information, systems, or physical spaces. It ensures that only authorized people or processes get the right level of access, reducing security risks.
 - Ensures only verified users can access resources
 - Uses authentication + authorization to control permissions
 - Protects both digital systems and physical spaces
 - Supports security, compliance, and accountability

Types of Access Control



- **1. Discretionary Access Control (DAC)**
- Access is controlled by the owner of the resource.
- Owners decide who gets access
- Flexible but less secure
- Common in personal computers and small systems
- **2. Mandatory Access Control (MAC)**
- A central authority enforces access based on strict security levels.
- Used in high-security environments
- Users cannot change permissions
- Example: Military or government systems, SELinux

- **3. Role-Based Access Control (RBAC)**

- Users receive permissions based on their job roles.
- Simplifies large-scale permission management
- Reduces human error and misuse
- Example: HR role cannot create network accounts

- **4. Attribute-Based Access Control (ABAC)**

- Access decisions are based on multiple attributes (user, device, environment).
- Highly dynamic and flexible
- Evaluates policies using many attributes
- Useful in modern cloud and zero-trust systems

- **5. Rule-Based Access Control (Ru-BAC)**

- Access is granted based on predefined rules or conditions.
- Often time-based or context-based
- System automatically enforces rules
- Example: Students allowed lab access only during class hours

- **6. Organization-Based Access Control (Or-BAC)**

- Access policies are defined independently of the technical system.
- Focuses on organizational rules
- Separates policy design from implementation
- Useful for complex enterprise environments

Categories of Access Control

- **1. Physical Access Control**
 - Controls entry to physical spaces like buildings and rooms.
 - Uses badges, keycards, locks, biometrics
 - Protects hardware and physical assets
 - Prevents unauthorized onsite access
- **2. Logical Access Control**
 - Controls access to digital resources like networks, systems, and data.
 - Uses passwords, MFA, firewalls, permissions
 - Protects sensitive digital information
 - Enforced through authentication and authorization

Email Security

- Email security protects your messages and data from unauthorized access, phishing, spam, and malware.
- It keeps your emails private, accurate, and safe using tools like encryption and spam filters.
- It protects against threats like phishing, spam, and malware using tools like encryption, spam filters, and two-factor authentication (2FA).
- By combining tech solutions and smart habits, you can ensure your emails stay secure.

Key Goals:

- **Privacy:** Only the intended recipient reads your email.
- **Accuracy:** Messages aren't tampered with.
- **Safety:** No viruses or scams sneak through.

Types of Email Attacks

- **Phishing**
- Phishing is a type of fraud where attackers impersonate trusted people or organizations—often by email, instant message, or social media—to trick victims into revealing credentials, financial data, or installing malware.
- Typical phishing messages mimic banks, vendors, or colleagues and use urgent language, fake links, or malicious attachments to prompt immediate action.

- **Vishing**

- Vishing (voice phishing) uses phone calls or voicemail to trick people into revealing sensitive information.
- Attackers spoof caller ID, use VoIP or prerecorded messages, and pose as banks, IT support, or colleagues to ask for verification codes, credit-card details, or remote-access permission.
- Because callers sound “real” and phone lines feel trusted, vishing often convinces victims to act quickly; targeted variants research the victim to make the call more believable.

- **Smishing**
- Smishing is phishing using text messaging on mobile phones.
- Criminals impersonate a legitimate source in an attempt to gain the trust of the victim.
- For example, a smishing attack might send the victim a website link.
- When the victim visits the website, malware is installed on the mobile phone.

Whaling

- The [Whaling](#) is a phishing attack that targets high-profile targets within an organization such as senior executives.
- Additional targets include politicians or celebrities.
- Whaling is a type of spear phishing attack that targets high-level executives or other high-value targets within an organization.
- These emails are designed to look like they come from a senior executive, such as the CEO or CFO, and often involve requests for money transfers or other sensitive information.
- One advantage of this attack is that it can be highly lucrative for the attacker, as they can potentially steal large sums of money or valuable information.

Pharming

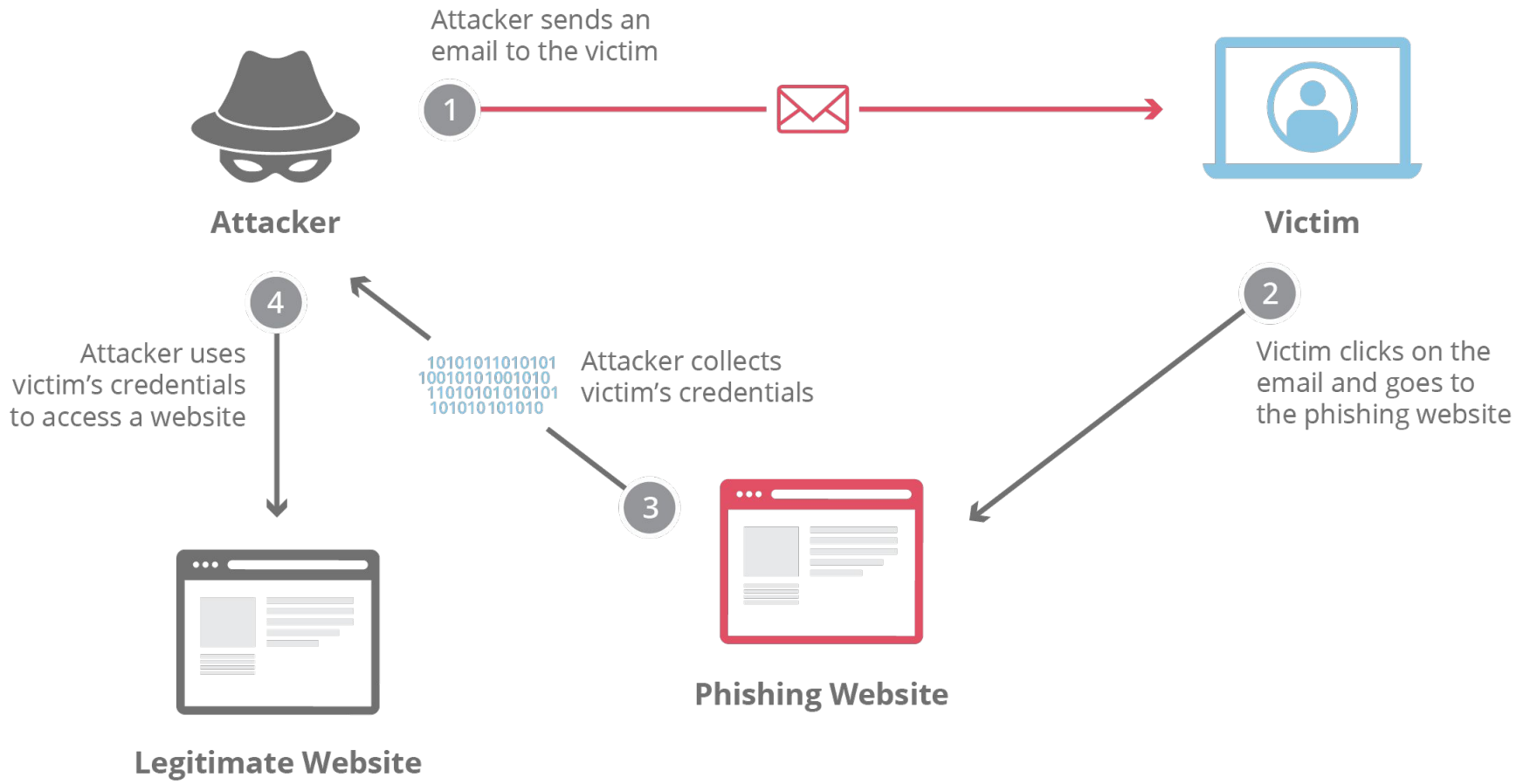
- Pharming is the impersonation of an authorized website in an effort to deceive users into entering their credentials.
- Pharming misdirects users to a fake website that appears to be official.
- Victims then enter their personal information thinking that they are connected to a legitimate site.

Spyware

- Spyware is software that enables a criminal to obtain information about a user's computer activities.
- Spyware often includes activity trackers, keystroke collection, and data capture.
- In an attempt to overcome security measures, spyware often modifies security settings.
- Spyware often bundles itself with legitimate software or with Trojan horses.
- Many shareware websites are full of spyware.

Phishing

- Phishing is a kind of cyber attack that is used to **steal users' information** including login details and credit card numbers.
- Most of the data breaches involve scams seeking to steal people's sensitive information or login credentials, which is a type of phishing attack.
- A phishing attack can be carried out with the help of fake emails cloning legitimate websites and tricking the user into revealing sensitive information.



Types of Phishing Attacks

Type of Phishing	Description
Email Phishing	Attackers send fraudulent emails pretending to be from legitimate sources to steal sensitive information.
Whaling	Targeted phishing aimed at high-profile individuals like CEOs or CFOs to steal company-sensitive data.
Vishing (Voice Phishing)	Attackers use phone calls to impersonate trusted entities, asking for sensitive information like account details.
Smishing	Phishing through SMS, often with links that lead to malicious websites or malware downloads.
Angler Phishing	Phishing via social media, where attackers impersonate customer service accounts to deceive users into visiting malicious websites.
HTTPS Phishing	Fake websites with HTTPS encryption that mislead users into thinking they are secure, collecting sensitive information like login details.
Spear Phishing	Highly targeted phishing using personalized information about the victim to make the attack appear legitimate.
Pharming	An attack that redirects a victim's traffic to a fake website by altering the DNS or browser settings.
Pop-up Phishing	Malicious pop-ups appear on legitimate websites, tricking users into clicking links or entering sensitive information.
Clone Phishing	Attackers replicate a previously sent legitimate email, replacing links or attachments with malicious content.
Evil Twin	A fake Wi-Fi hotspot set up to steal sensitive data from users who connect to it.

Protecting Against Email Attacks

- Never provide your personal information in response to an unsolicited request.
- If you believe the contact may be legitimate, contact the financial institution yourself.
- Never provide your password over the phone or in response to an unsolicited Internet request.
- Review account statements regularly to ensure all charges are correct.

OWASP Top 10 Vulnerabilities

- OWASP stands for the Open Web Application Security Project.
- It is a non-profit global online community consisting of tens of thousands of members and hundreds of chapters that produces articles, documentation, tools, and technologies in the field of web application security.

- OWASP releases the Top 10 Web Application Security Risks every 3-4 years based on real-world vulnerability data.
- The list is created using frequency, severity, and impact of security flaws found in web applications.
- It highlights the most common and most dangerous vulnerabilities that attackers frequently exploit.
- The latest update (2021) helps developers and security professionals build more secure applications.
- It is considered an essential security guide and industry standard for web application security best practices.

OWASP TOP 10

Broken Access Control

1

Cryptographic Failures

2

Injectons

3

Insecure Design

4

Security Misconfigurations

5

Vulnerable and Outdated
Components

6

Identification and
Authentication Flaws

7

Software and Data
Integrity Failure

8

Security Logging and
Monitoring Flaws

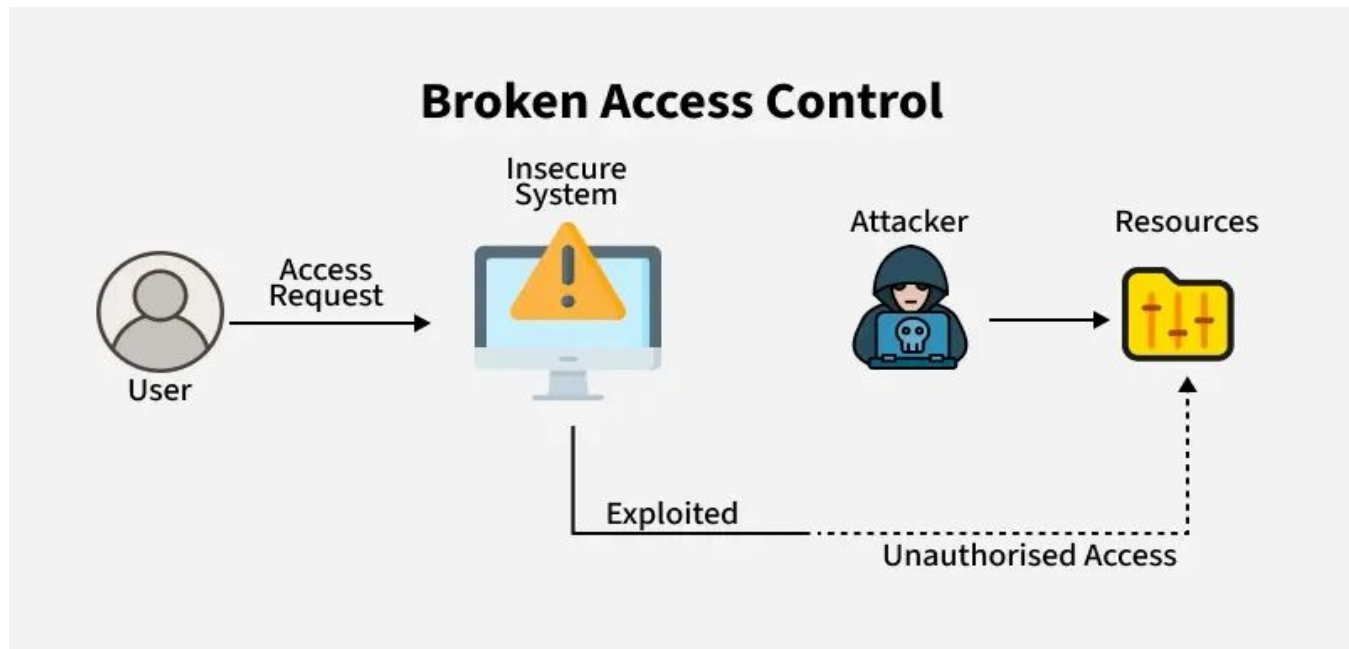
9

Server-Side Request
Forgery (SSRF)

10

1. Broken Access Control

- This vulnerability occurs when access controls are poorly enforced, allowing authenticated users to reach data or functions they shouldn't, such as other users' accounts or confidential files.
- Attackers often exploit weak session management or unexpired session tokens to gain unauthorized access to valid accounts.



- **Vulnerabilities under Broken Access Control**

- A system where access is not granted as per user roles, i.e. anyone in the system can access any resource, no principle of least privilege is being followed.
- Use of insecure direct object references to access someone else's account without their knowledge
- Lack of access controls in PUT, POST, DELETE in APIs
- Any kind of tampering with JSON web tokens to elevate privileges like changing roles from 'user' to 'admin'.

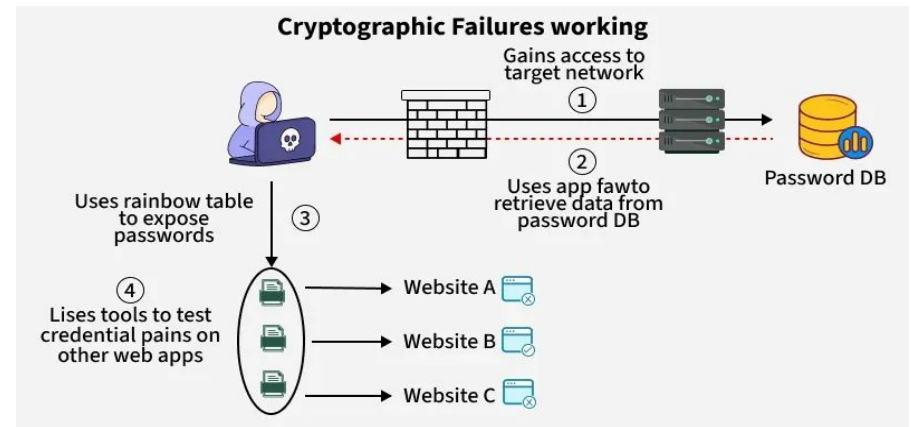
- **Example of Broken Access Control Vulnerability**
- If an application used the URL of the format:
- [<http://example.com/object/12345%3C/span>](http://example.com/object/12345%3C/span)
- The reference ID 12345 could be changed to 123456 and the user could simply access the other object with that id.
- **Broken access control attacks can be prevented by**
 - Use role-based or attribute-based access control.
 - Always enforce authorization on the server side.
 - Centralize access control logic.
 - Verify resource ownership before access.
 - Avoid exposing internal IDs directly (prevent IDOR).
 - Log and monitor access control failures.
 - Apply the principle of least privilege.

2. Cryptographic Failures

- Any organization's data, whether in transit or at rest, is vulnerable to cryptographic failure if it is not properly secured.
- Sensitive information such as credentials, health records, classified data, and business secrets may be exposed.
- If this data is protected under privacy laws or regulations, its exposure due to weak or missing encryption is considered a cryptographic failure.

Vulnerabilities under Cryptographic Failures

- Old or weak cryptographic measures being used by default in a system.
- Data transmission without being encrypted, use of protocols such as HTTP, SMTP, FTP.
- Use of unapproved hash functions such as MD5 or SHA1
- Server certificate and trust chain is not verified properly
- Use of default crypto keys, weak crypto keys, keys being re-used, improper key management in a system



- **Example of a Cryptographic Failure**

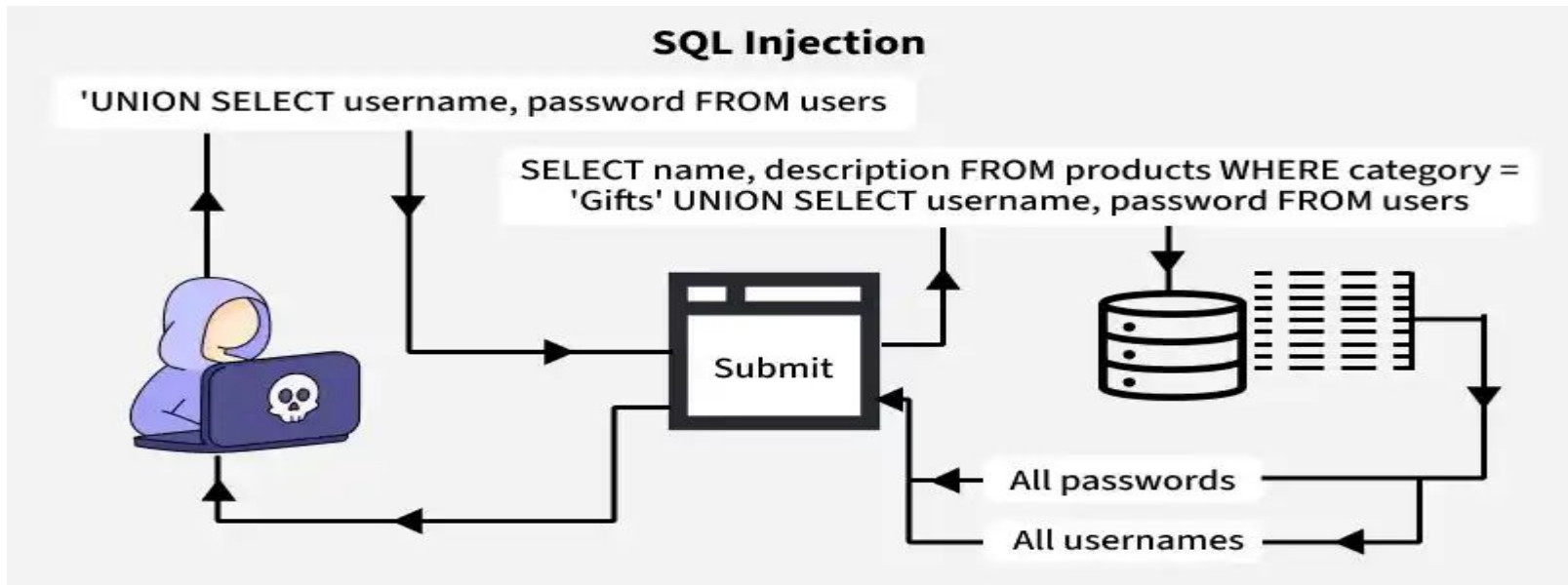
- A company stores the passwords of its users without proper encryption.
- If an attacker successfully gains access to the database, could easily gain credentials of all the users.
- Hashes that are made by simple hash functions could be easily cracked by strong GPUs.

Ways to Prevent Cryptographic Failures

- Use strong, modern encryption algorithms like AES-256 to protect sensitive data.
- Rotate cryptographic keys regularly to reduce the risk of exposure.
- Generate keys using secure, random processes to avoid predictability.
- Implement multi-factor authentication (MFA) for accessing encryption systems.
- Ensure encryption of both data at rest and in transit to safeguard information throughout its lifecycle.
- Audit cryptographic systems regularly to detect weaknesses and vulnerabilities.
- Avoid hard-coding keys in code, store them in secure, external locations.

3. Injection

- Injection vulnerabilities occur when an attacker uses a query or command to insert untrusted data into the interpreter via SQL, OS, NoSQL, or LDAP injection.
- The data that is injected through this attack vector makes the application do something it is not designed for.
- Not all applications are vulnerable to this attack, only the applications that accept parameters as input are vulnerable to injection attacks.



Causes of Injection Vulnerabilities

- When the data input by users is not verified, sanitized or filtered.
- It directly adds user input into SQL or command strings, which mixes trusted code with untrusted data.
- It uses user input in ORM queries, letting attackers fetch data they shouldn't access.
- It builds database queries using user input without using parameters or escaping, which can lead to injections.

Example of an Injection Vulnerability

- The code for querying in the backend of the application
- `SELECT * FROM users WHERE username = '$username' AND password = '$password';`The attacker inputs the username and password as
- Username: admin Password: ' OR '1'='1'Now, the resultant query will be
- `SELECT * FROM users WHERE username = 'admin' AND password = ' OR '1'='1';`Given that the condition '1'='1' is true, the query will be successfully carried out and the user gained access to system.

Injection attacks can be prevented by

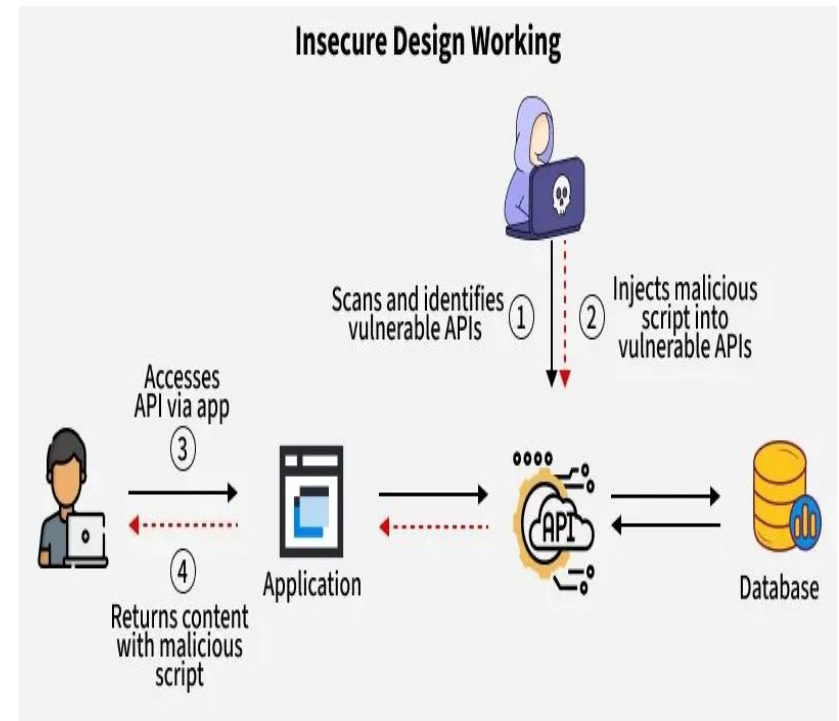
- Use parameterized queries or prepared statements.
- Validate and sanitize all user inputs.
- Avoid building SQL queries using string concatenation.
- Apply the principle of least privilege to database users.
- Keep your database and libraries up to date.
- Deploy a Web Application Firewall (WAF) as an extra layer.

4. Insecure Design

- Insecure Design vulnerabilities refer to weaknesses that lie in the designing process of a product.
- These weaknesses cannot be overcome by secure implementation.
- They include flaws like lack of assessment of security requirements during the design phase.
- Principles like Secure design, SDLC, Resource management are overlooked or not properly configured during development phase.

Example of Insecure Design Vulnerability

- A mobile banking app allows users to transfer money between accounts.
- However, the app's design does not require re-authentication for high-value transactions, like transferring a large sum of money. Instead, it only asks for authentication during the initial login session.
- An attacker who gains access to a user's unlocked phone can initiate a high-value transfer without needing to re-authenticate, allowing unauthorized transactions.

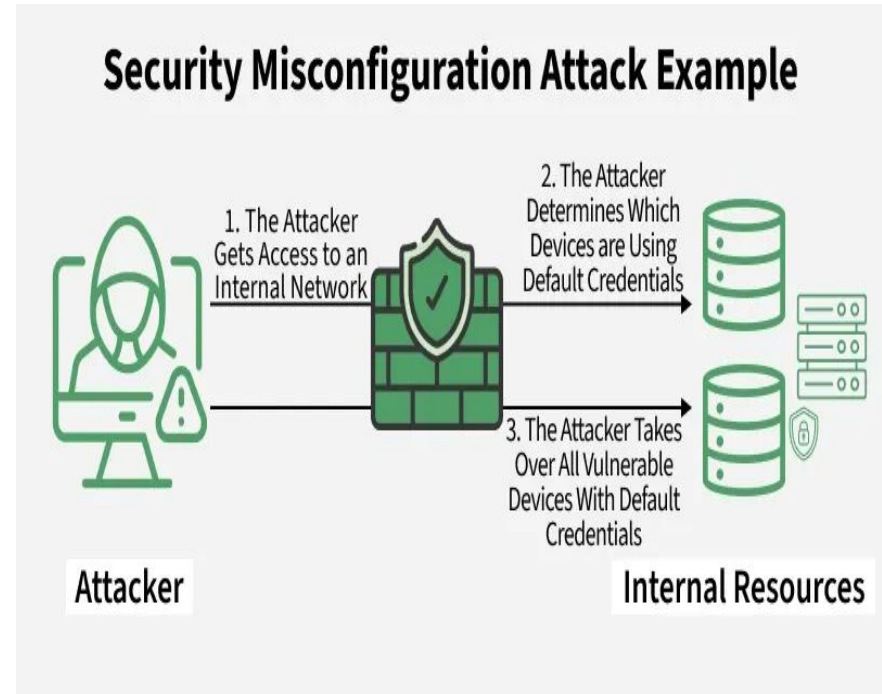


How to Prevent Insecure Design Vulnerability

- Use Secure development lifecycle during the designing phase of a product
- A pre-configured library of secure design patterns to use as a roadmap
- Threat modeling for designing authentication, access controls, business logics and key flows
- Conduct unit and integration tests to check if all critical flows of the design are safe as per the threat model
- A comprehensive document containing use-cases and misuse-cases for each level of the application
- Segregate the tier layers and network layers on the system according to the level of protection needed for each of them

5. Security Misconfiguration

- It is estimated that up to 95% of cloud breaches are the result of human errors and this fact leads us to the next vulnerability called security misconfiguration.
- This vulnerability refers to the improper implementation of security intended to keep application data safe.



Vulnerabilities covered under Security Misconfiguration

- Use of default credentials by an account
- Latest security features on disabled mode in the updated systems
- Outdated software
- Insecure security settings like libraries, databases, application frameworks
- Lack of appropriate security measures in the cloud services
- Security headers not set to secure values

Example of Security Misconfiguration

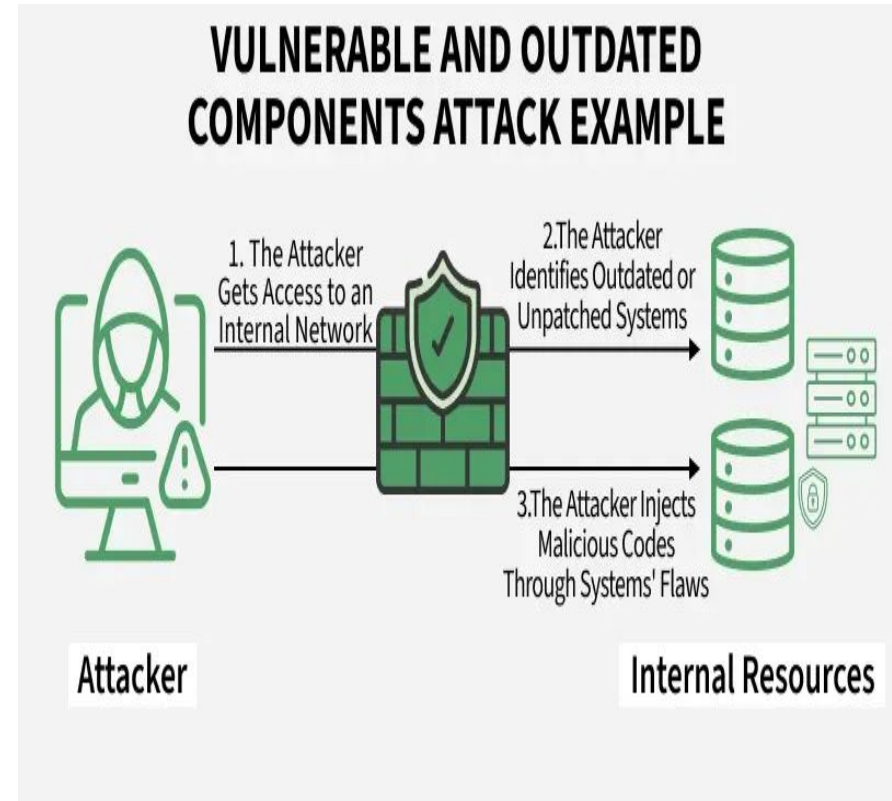
- An attacker scans public-facing sites for common CMS platforms and attempts to log in using known default credentials.
- The attacker successfully logs in to the admin panel and gains full control of the website, including access to user data and the ability to upload malicious content.

How to Prevent Security Misconfiguration Vulnerabilities

- Using Dynamic application security testing (DAST)
- Disabling the use of default passwords
- Keeping an eye on cloud resources, applications, and servers
- Automated process to verify the effectiveness of security configurations time to time

6. Vulnerable and Outdated Components

- Nowadays there are many open-source and freely available software components (libraries, frameworks) that are available to developers and if there occurs any component which has got a known vulnerability in it then it becomes a weak link that can impact the security of the entire application.
- Attackers can take advantage of these weaknesses to gain unauthorized access, execute code, or disrupt application functionality.



Vulnerabilities due to Vulnerable and Outdated Components

- Lack of information about all the components including client and server side, including direct and nested dependencies
- Vulnerable or outdated software, including databases, OS, servers, DBMS, APIs, runtime environments, libraries and all other components of an application.
- Irregular scan of vulnerabilities in a system
- Untested compatibility of updated, upgraded and patched libraries by software developers

Example of Vulnerable and Outdated Components

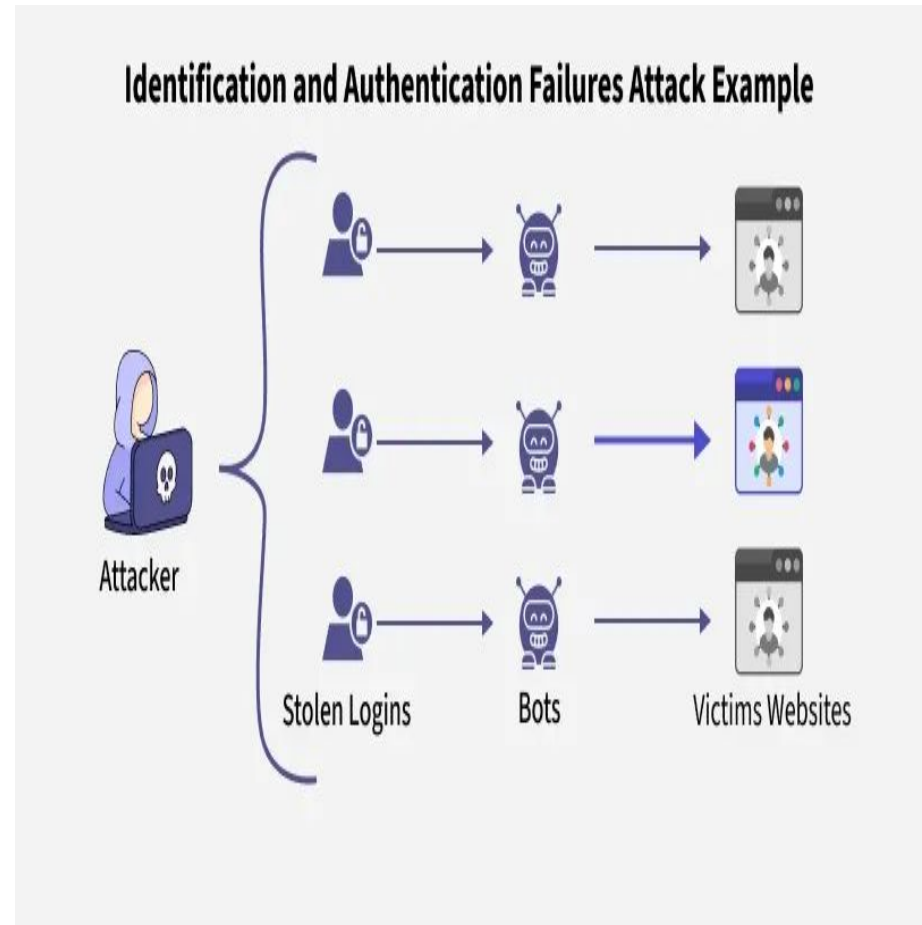
- A company's web application runs on Apache Struts 2.3, a version with a known vulnerability that allows remote code execution (RCE). Despite the patch being available for months, the company fails to update the framework.
- An attacker scans for applications using this outdated version, exploits the vulnerability, and gains control of the server allowing them to execute arbitrary commands, access sensitive data, and potentially pivot deeper into the network.

Vulnerable and Outdated Components attacks can be prevented by

- Remove unnecessary dependencies, features, components and files
- Install components of a system only from official sources through secure channels only.
- Properly maintain the libraries and components and regularly check for updates and upgrades for each.

7. Identification and Authentication Failures

- his vulnerability was previously listed as Broken Authentication, It is a vulnerability that allows an attacker to use manual or automatic methods to try to gain control over any account they want in a system. In worse conditions, they could also gain complete control over the system.



Causes of Identification and Authentication Failures

- Automated attacks like credential stuffing, brute-force attacks to gain unauthorized access to a system
- Default and weak credentials are allowed by any system
- Lack of Multi-factor authentication
- Insecure password recovery mechanisms
- Unencrypted storage of usernames and passwords
- Improper validation of Session IDs

Example of Identification and Authentication Failure

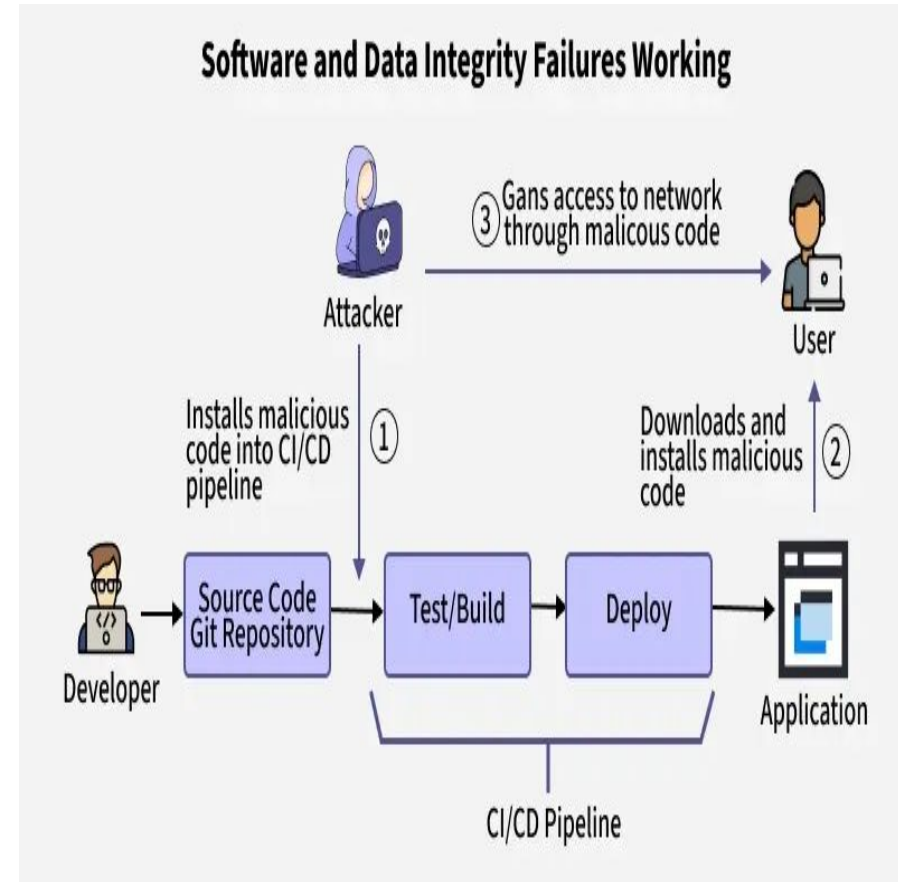
- A mobile app allows users to log in using only their email address, without requiring a password or any second factor of authentication.
- Once an attacker identifies a valid email address, they can impersonate that user and gain full access to their account without needing to prove their identity.

Identification and Authentication Failures can be prevented by

- Implementing multi-factor authentication
- Protecting user credentials
- Sending passwords over encrypted connections
- Weak passwords should not be allowed for any user
- Credential Recovery process must be secured

8. Software and Data Integrity Failures

- When the integrity of a software or data could be compromised by an attacker leads to software and data integrity failures.
- If an application relies on dependencies like libraries, modules or plugins from a untrusted source or repository it could lead to Software and Data Integrity Failures.
- Auto-update functionality where systems are updated automatically without a proper integrity verification are vulnerable to attacks.



Vulnerabilities under Software and Data Integrity Failures

- Untrusted sources of dependencies for plugins, libraries, modules
- Insecure CI/CD pipeline leading to unauthorized access, malware upload or system compromise
- Auto-update without proper integrity verification

Example of Software and Data Integrity Failures

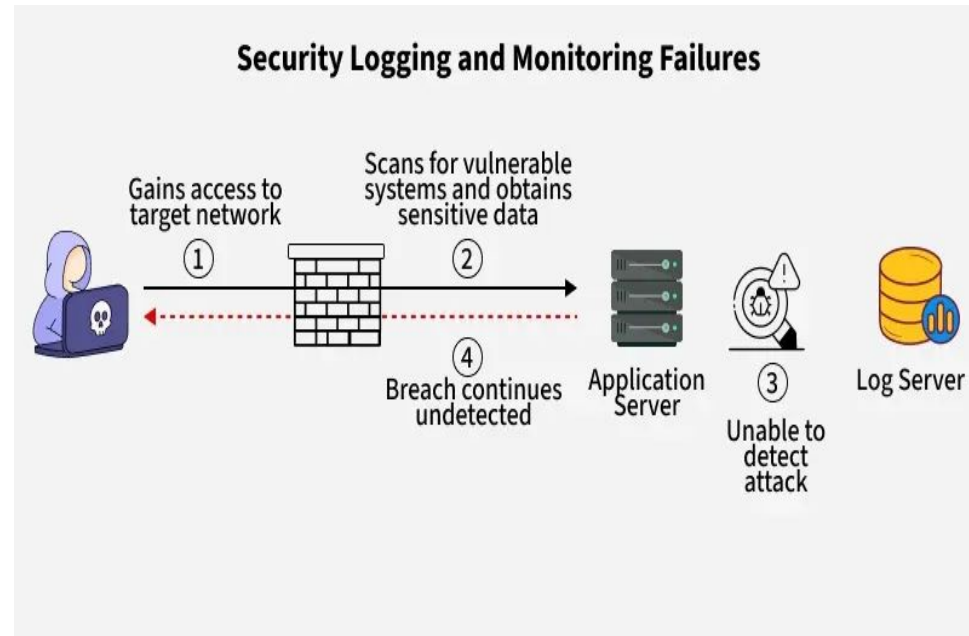
- A company sets up automatic updates for its desktop software but does not verify the authenticity of the update source or use digital signatures.
- An attacker compromises the update server and pushes a malicious software update. All clients that download the update unknowingly install malware.

Software and Data Integrity Failures can be prevented by

- Ensuring libraries and dependencies are installed from trusted repositories
- Unencrypted serialized data should not be sent to untrusted clients without an integrity check
- Use of digital signature to verify the integrity of any software or data
- Efficient code review process for code and configuration changes to reduce the chance of injected malicious code into the software pipeline

9. Security Logging and Monitoring Failures

- Security Logging and Monitoring Failures occur when applications do not properly log critical events or fail to monitor and alert on suspicious activities.
- This can delay detection of breaches, hinder incident response, and allow attackers to operate undetected within systems.



Example of Security Logging and Monitoring Failure

- An e-commerce website suffers a brute-force login attack, where an attacker makes thousands of login attempts using different username and password combinations.
- However, the system lacks proper logging and monitoring, so the repeated failed login attempts go unnoticed by the security team until user accounts are compromised.

Security Logging and Monitoring

Failure can be prevented by

- Login controls, access controls and server-side input validation must be ensured
- Logs generated by the system should follow a particular format that can be easily stored and processed by log management solutions
- Effective monitoring and flagging of suspicious activities that are responded addressed
- A proper implementation of an incident response plan in case of security incident

10. Server-Side Request Forgery

- SSRF (Server-Side Request Forgery) is a newly added vulnerability to the list of OWASP-10, it refers to when a web application does not validate the user-supplied URLs before fetching them, which lets the attacker force the legit website to send a forged request to an unexpected destination, despite being protected by firewalls, access controls etc.



Example of Server-Side Request Forgery

- A web application allows users to input a URL to fetch and display metadata (e.g., a preview of a link).
- The server takes the user-supplied URL and makes an HTTP request to retrieve content, without proper validation or restriction.
- An attacker enters the following URL:
- <http://169.254.169.254/latest/meta-data/iam/security-credentials/>
- This is the internal metadata service IP used by cloud providers like AWS. The server, running in a cloud environment, fetches the internal URL and returns sensitive cloud instance credentials to the attacker.
- The attacker tricked the server into making a request to an internal resource that would otherwise be inaccessible, leading to credential theft or internal network access

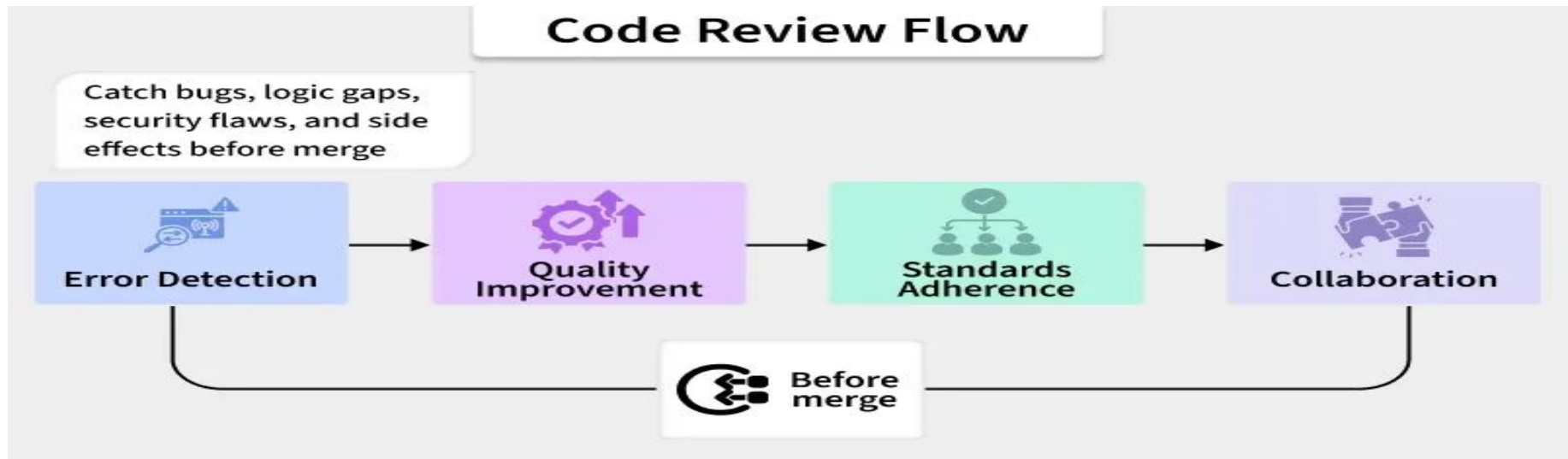
Server-Side Request Forgery can be prevented by

- Sanitization and validation of all client-side input data
- HTTP redirections should be disabled
- Avoid using server-side functionality to fetch remote URLs unless absolutely necessary.
- If URL fetching is required, limit it to internal logic with strict controls.
- Use firewalls and network policies to prevent outbound requests to internal or sensitive systems.

Secure Code Review

- A Secure Code Review is a systematic evaluation of application source code (and related build/config) to find, prioritise, and remediate security weaknesses before they reach production.
- **It Purpose to:**
 - Find vulnerabilities early (logic flaws, injection, auth bugs, secrets).
 - Reduce production risk and attack surface.
 - Link findings to business impact and remediation plans.

What is Code Review?



- Code review is a systematic examination of source code by peers to detect bugs, improve readability and maintainability, verify standards and best practices, strengthen security, and share knowledge.
- Performed before merging into a feature or main branch, it provides a second opinion on solution and implementation, helps manage and mitigate risk, and supports building “self-defending” applications when woven into the SDLC with support from secure-coding SMEs and review tools.

Why is Secure Code Review Important?

- Secure code review plays a crucial role in enhancing the security posture of software applications.
- It helps identify security vulnerabilities such as injection flaws, authentication issues, and insecure configurations early in the development lifecycle.
- By detecting and addressing these issues before deployment, organizations can reduce the risk of security breaches and ensure compliance with regulatory standards.
- **It offers several key benefits:**
 - **Proactive Risk Mitigation:** Early identification of these openings mean that the organizations can avoid costly system breakages and loss of data.
 - **Improved Software Quality:** Secure code review contributes to making the code more secure, thus improving the quality of the developed software.
 - **Compliance Adherence:** Numerous trades have the tendency of having strict security policies. Secure code review assists an organization to deliver on compliance needs.
 - **Enhanced Developer Skills:** Daily code reviews enhance the organization's security posture and the proficiency of the coding team.

What are the Elements of Secure Code Review?

- **Static Analysis Tools:** Automated tools scan the source code to identify common vulnerabilities and coding errors.
- **Manual Review:** Experienced developers and security experts manually inspect the code for logic flaws, design issues, and vulnerabilities that automated tools might miss.
- **Security Guidelines:** Following established security coding guidelines (e.g., OWASP Top 10) ensures that developers adhere to secure coding practices during code review.

- **Peer Review:** Collaboration among team members allows for knowledge sharing and ensures that potential security issues are identified and addressed collectively.
- **Code Analysis:** By applying static and dynamic analysis tools and using source or binary code, the tool is able to detect possible openings on its own.
- **Manual Review:** People in identifying other problems that could be easily overlooked by the bot analyzing code.
- **Threat Modeling:** Evaluating risks and threats affecting the dispersed application.

- **Coding Standards and Guidelines:**
Implementing the codification structure that has been prescribed and approved security protocols.
- **Vulnerability Remediation:** Identifying appropriate measures to counter existing weak spots.